

Lab 3

```
In [ ]: # Definición de Las matrices como Listas de Listas
M = [[120, 5],
      [5, 115]]

Mp = [[200, 10],
       [10, 1]]

M, Mp
def det2x2(A):
    return A[0][0]*A[1][1] - A[0][1]*A[1][0]

def trace2x2(A):
    return A[0][0] + A[1][1]
```

```
In [3]: # Cálculo manual de eigenvalores para una matriz 2x2
def eigenvalues_2x2(A):
    a = 1
    b = -(A[0][0] + A[1][1])
    c = det2x2(A)

    discriminant = b*b - 4*a*c
    sqrt_disc = discriminant ** 0.5

    lambda1 = (-b + sqrt_disc) / (2*a)
    lambda2 = (-b - sqrt_disc) / (2*a)

    return lambda1, lambda2
```

```
In [4]: eig_M = eigenvalues_2x2(M)
eig_M
eig_Mp = eigenvalues_2x2(Mp)
eig_Mp
```

Out[4]: (200.50124999218758, 0.4987500078124043)

Como visto en clase:

R = det(M) – k · (trace(M))²

```
In [6]: k = 0.04

det_M = det2x2(M)
trace_M = trace2x2(M)

R_M = det_M - k * (trace_M ** 2)

det_M, trace_M, R_M
```

Out[6]: (13775, 235, 11566.0)

```
In [7]: det_Mp = det2x2(Mp)
trace_Mp = trace2x2(Mp)

R_Mp = det_Mp - k * (trace_Mp ** 2)

det_Mp, trace_Mp, R_Mp
```

Out[7]: (100, 201, -1516.04)

Para la matriz M se obtuvo:

det(M) = 13775
trace(M) = 235
R = 11566

El valor de R es positivo y significativamente grande. Según el criterio de Harris, cuando R > 0 y los eigenvalores son grandes y similares, el pixel corresponde a una ESQUINA, ya que existen cambios de intensidad importantes en ambas direcciones.

Clasificación según eigenvalores y Harris

→ Representa una ESQUINA si:

- Eigenvalores grandes y similares
- Respuesta de Harris positiva

→ Representa un BORDE si:

- Un eigenvalor grande y uno pequeño
- Respuesta de Harris negativa

Task 3 — Análisis crítico (basado en datos empíricos)

Resumen de mediciones (promedios)

Algoritmo	Tiempo total (ms)	Det+Desc (ms)	Matching (ms)	Keypoints (A/B)	Good Matches (post-ratio)	FPS aprox (1000/tiempo)
SIFT	836.92	664.28	172.63	4268 / 4604	308	1.2
ORB	66.44	50.14	16.31	2000 / 2000	30	15.1

Referencia: 60 FPS ≈ 16.67 ms de presupuesto total por frame.

3a) ¿Cuál algoritmo elegiría para el Producto A (Drone de Carreras) y por qué?

Se elegiría ORB para el Producto A, porque es el único que se acerca a un escenario de tiempo real dentro de las dos opciones evaluadas.

- El producto requiere **60 FPS mínimo**, lo que implica un presupuesto total de ~**16.67 ms** por frame.
- Según las mediciones:
 - SIFT = 836.92 ms** → ~**1.2 FPS**, está completamente fuera del rango para 60 FPS.
 - ORB = 66.44 ms** → ~**15.1 FPS**, todavía **no alcanza 60 FPS**, pero es **mucho más cercano** y es la alternativa con mayor potencial de optimización.

Trade-off técnico para el caso A (tiempo real + blur):

- En un drone de carreras con **imagen borrosa y de baja calidad**, se prioriza un algoritmo rápido, aunque sacrifique parte de la robustez o densidad de correspondencias.
- ORB produce menos correspondencias que SIFT (**30 good matches vs 308**), pero el costo computacional es significativamente menor (**66.44 ms vs 836.92 ms**).
- ORB permite aplicar estrategias para acercarse al objetivo de 60 FPS, por ejemplo:
 - reducir resolución o usar pirámides con niveles más agresivos,
 - limitar región de interés (ROI) y evitar detectar en toda la imagen,
 - reducir nfeatures ,
 - usar procesamiento paralelo / optimizaciones de hardware (SIMD / GPU),
 - aplicar un pipeline de odometría visual más incremental (no recomputar todo desde cero cada frame).

Conclusión (A):

- SIFT se descarta** para el Producto A por exceder drásticamente el presupuesto de tiempo.
- ORB es la decisión correcta** entre ambas opciones por su menor tiempo de ejecución, aunque requeriría optimizaciones para llegar a 60 FPS.

3b) ¿Cuál algoritmo elegiría para el Producto B (Inspección) y por qué?

Se elegiría SIFT para el Producto B, porque el tiempo no es crítico y el producto exige **precisión de emparejamiento milimétrica** para stitching/registro.

- El Producto B toma imágenes de alta resolución (4K) y busca construir un mapa detallado; en este contexto se valora:
 - robustez ante cambios de escala y rotación**, y
 - densidad/calidad de correspondencias**, ya que una mayor cantidad de matches confiables mejora la estimación geométrica (p.ej., homografía o modelos más complejos).

Evidencia empírica (métricas):

- SIFT detectó más keypoints (**4268/4604**) que ORB (**2000/2000**).
- SIFT obtuvo más matches “buenos” post-ratio test (**308**) que ORB (**30**).
- En la etapa del Task 2, SIFT produjo más inliers post-RANSAC (**156**) que ORB (**18**), lo cual indica mayor consistencia geométrica global.

Análisis visual (cambios de escala y rotación):

- En la prueba con **rotación (~45°) y cambio evidente de escala**, SIFT mantuvo un conjunto de correspondencias abundante y consistente sobre la región texturizada del objeto, lo cual es favorable para stitching preciso.
- ORB no falló completamente (sí logró correspondencias), pero mostró una degradación clara en cantidad de matches, lo cual puede reducir precisión y estabilidad de la transformación estimada al haber menos restricciones geométricas.

Conclusión (B):

- SIFT es preferible** para el Producto B por su mejor desempeño en robustez y cantidad/calidad de correspondencias en presencia de cambios de escala y rotación, a pesar de ser considerablemente más lento.

3c) ¿Las conclusiones son justas y generalizables? ¿Por qué? ¿Qué considerar en futuras iteraciones?

Las conclusiones son válidas para el experimento realizado, pero no completamente generalizables, por lo siguiente:

- Solo se evaluó un par de imágenes y una escena**
 - El desempeño puede cambiar con distinta textura (poca textura vs mucha), iluminación, oclusiones, brillo/especularidad, y diferentes patrones repetitivos.
- El hardware y la configuración influyen en los tiempos**
 - Los milisegundos medidos dependen del CPU, del build de OpenCV (optimizaciones), y de si existe aceleración (SIMD / GPU).
 - Además, parámetros como `nfeatures` en ORB o configuraciones internas de SIFT modifican tiempos y resultados.
- La medición corresponde a “detección+descripción” y “matching” en batch**
 - En un sistema real de odometría visual, típicamente se usan pipelines incrementales (frame-to-frame) y técnicas adicionales (tracking, keyframes), lo cual cambia el costo real por frame.
- La calidad de matching no se evaluó con métricas de error geométrico**
 - Se midieron keypoints y matches, pero no se cuantificó el error final (p.ej., reprojection error promedio, drift en odometría, error de alineación en stitching).
 - Para “precisión milimétrica”, sería ideal validar con métricas geométricas en un set de pruebas controlado.

Qué se debería considerar en futuras iteraciones

- Probar múltiples escenas (alta/baja textura) y múltiples condiciones (blur, ruido, cambios de iluminación).
- Repetir mediciones a distintas resoluciones (simular baja calidad para A y 4K para B).
- Medir precisión con métricas geométricas (reprojection error, error de homografía, error de stitching).
- Ajustar hiperparámetros de cada algoritmo y reportar trade-offs (tiempo vs matches vs error).
- Evaluar alternativas para tiempo real y blur (p.ej., variantes rápidas, tracking por optical flow + keyframes) y alternativas para máxima precisión (p.ej., refinamiento subpíxel, bundle adjustment en stitching).

Conclusión final (decisión por producto)

- Producto A (Drone de Carreras, 60 FPS, blur):** se selecciona **ORB** por velocidad, siendo el único cercano a tiempo real entre ambos.
- Producto B (Inspección 4K, stitching, precisión):** se selecciona **SIFT** por robustez y mayor cantidad/calidad de correspondencias bajo cambios de escala y rotación.